DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

Student Name: Kaviya Umapathy

Register Number: 1965260209

Department: Computer Science and Engineering (Cybersecurity)

Total Marks: 50

Submission Date: 21 October 2026

Q1 – Sorting Under Different Data Conditions (6 Marks)

1. Prediction and algorithmic reason

Selection sort performs n(n–1)/2 comparisons regardless of input order and makes at most n–1 swaps. Optimised bubble sort can stop early when no swap occurs. Insertion sort is adaptive: it performs little work on sorted or nearly sorted data, but many shifts on reverse-sorted data. All three have $O(n^2)$ worst-case time; insertion sort is O(n) on already sorted input.

2. Designed datasets

Dataset A – already sorted: [1, 2, 3, 4, 5, 6]. **Dataset B – reverse sorted:** [6, 5, 4, 3, 2, 1]. **Edge case C:** [7], a single element. The first dataset represents the common ordered condition; the second exposes movement cost; the edge case tests minimum input size.

3. Implementations

```
def selection_sort(a):
    a = a[:]; comparisons = swaps = 0
    for i in range(len(a)-1):
        m = i
        for j in range(i+1, len(a)):
            comparisons += 1
            if a[j] < a[m]: m = j
        if m != i:
            a[i], a[m] = a[m], a[i]; swaps += 1
    return a, comparisons, swaps

def bubble_sort(a):
    a = a[:]; comparisons = swaps = 0
    for end in range(len(a)-1, 0, -1):
        changed = False
        for j in range(end):
            comparisons += 1
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                swaps += 1; changed = True
        if not changed: break
    return a, comparisons, swaps

def insertion_sort(a):
    a = a[:]; comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]; j = i-1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j+1] = a[j]; shifts += 1; j -= 1
            else: break
        a[j+1] = key
    return a, comparisons, shifts
```

4. Operation-count evidence

Input (n=6)	Algorithm	Key comparisons	Swaps / shifts
Sorted	Selection sort	15	0 swaps
Sorted	Bubble sort (early stop)	5	0 swaps
Sorted	Insertion sort	5	0 shifts
Reverse	Selection sort	15	3 swaps
Reverse	Bubble sort	15	15 swaps
Reverse	Insertion sort	15	15 shifts
Single element	All three	0	0

The operation counts agree with the prediction: selection sort does not benefit from input order, while bubble and insertion sort do. Execution time is omitted because it depends on the machine and is less reliable than operation counts.

5. Recommendation

Recommend insertion sort because about 95% of orders are already correctly ordered and only small batches arrive. It inserts new records into an ordered list with relatively few comparisons and shifts. Selection sort still scans the remaining suffix, while bubble sort is less effective for small updates. For very large datasets, evaluate a more scalable sorting method.

Q2 – When Is Sequential Search Appropriate? (4 Marks)

1. Experimental design

Search for a Tool ID at the first, middle and last positions, and search for a missing ID. Repeat with list sizes n = 5, 100 and 1,000. Count one comparison each time a stored Tool ID is checked.

2. Comparison-count evidence

List size	First item	Middle item	Last item	Not found
5	1	3	5	5
100	1	50	100	100
1,000	1	500	1,000	1,000

3. Interpretation and recommendation

Sequential search is reasonable for small, frequently changing lists when searches are infrequent or maintaining an index would add overhead. A successful search requires 1 to n comparisons; an unsuccessful search requires n. If every position is equally likely, average successful search cost is approximately (n+1)/2. Its time complexity is O(n).

After expansion to 20 facilities, active records and lookup requests will rise. For large lists and frequent queries, use a hash table keyed by Tool ID for expected O(1) lookup, or a balanced search tree for O(log n) lookup and ordered traversal. Updates must remain consistent when tools are issued or returned.

Q3 – Brute-Force String Matching (6 Marks)

1. Method and implementation

At every valid starting position, compare the pattern from left to right and stop at the first mismatch. A full match is reported when all pattern characters agree. For text length n and pattern length m, the worst-case comparison count is (n–m+1)m, giving O(nm) time.

```
def brute_force_match(text, pattern):
    comparisons = 0; positions = []
    n, m = len(text), len(pattern)
    if m == 0: return [0], 0
    for i in range(n-m+1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i+j] != pattern[j]: break
            j += 1
        if j == m: positions.append(i)
    return positions, comparisons
```

2. Predictions and operation counts

Test	Text	Pattern	Prediction / result
A – low work	abcdefghij	z	Immediate mismatch at each alignment: 10 comparisons
B – expensive	aaaaaaaaaa	aaaab	Four matching characters then mismatch at each of 6 alignments: 30 comparisons
C – repeated matches	abcabcabc	abc	Matches at positions 0, 3, 6; 7 valid alignments × 3 comparisons = 21

Correction to the sample calculation: for Test C, text length 9 and pattern length 3 give 7 valid alignments, each requiring 3 comparisons in this text, so the total is 21 comparisons. The algorithm performs more work when a long prefix repeatedly matches before a mismatch. Input size alone does not determine work; character distribution, prefix similarity and mismatch position also matter.

Q4 – Brute-Force Closest Pair (6 Marks)

1. Algorithm and mathematical pair count

Examine each unordered pair once using i = 0…n–1 and j = i+1…n–1. This avoids self-comparisons and duplicate pairs (i,j) and (j,i).

Number of unique pairs = (n–1) + (n–2) + … + 1 = n(n–1)/2.

```
def closest_pair(points):
    best_pair = None; best_d2 = float("inf")
    pairs = distances = 0
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            pairs += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            d2 = dx*dx + dy*dy
            distances += 1
            if d2 < best_d2:
                best_d2 = d2; best_pair = (points[i], points[j])
    if best_pair is None: return None, None, pairs, distances
    return best_pair, best_d2**0.5, pairs, distances
```

2. Growth evidence

Number of drones (n)	Unique pairs	Distance calculations
10	45	45
100	4,950	4,950
1,000	499,500	499,500

The pair count grows quadratically: doubling n produces about four times as many pairs. Define an acceptable response time as 1 second on the target computer and use a high-resolution monotonic timer to measure it. The exact practical limit must be measured on that machine; operation counts provide hardware-independent evidence.

3. Scalability

The inefficiency comes from calculating every pair, including pairs that cannot improve the current best distance. A redesigned strategy should organise points spatially and prune candidate pairs whose minimum possible distance exceeds the best distance found. A divide-and-conquer closest-pair algorithm can achieve O(n log n) time under standard assumptions.

Q5 – Brute-Force Convex Hull (7 Marks)

1. Geometric decision rule

For candidate points A=(x■,y■), B=(x■,y■), and another point P=(x,y), compute:

$$\text{cross}(A,B,P) = (x■ – x■)(y – y■) – (y■ – y■)(x – x■)$$

A positive value places P on one side of directed line AB; a negative value places it on the other; zero means collinear. If all remaining points lie on one side or on the line, AB is a supporting line and may form a hull edge. For collinear points, retain extreme endpoints and remove duplicate edges.

2. Pseudocode

```
hull_edges = []
for each pair of distinct points A, B:
    positive = False; negative = False
    for each point P other than A and B:
        c = cross(A, B, P)
        if c > 0: positive = True
        if c < 0: negative = True
    if not (positive and negative):
        record the supporting boundary edge
        resolve collinear points using extreme endpoints
order the accepted hull vertices around the boundary
```

3. Implementation

```
def cross(a, b, p):
    return ((b[0]-a[0])*(p[1]-a[1])
            - (b[1]-a[1])*(p[0]-a[0]))

def brute_force_hull(points):
    pts = list(dict.fromkeys(points))
    if len(pts) <= 2: return pts
    edges = set()
    for i, a in enumerate(pts):
        for j, b in enumerate(pts):
            if i == j: continue
            pos = neg = False
            for k, p in enumerate(pts):
                if k == i or k == j: continue
                c = cross(a, b, p)
                if c > 0: pos = True
                elif c < 0: neg = True
            if not (pos and neg):
                edges.add((a, b))
    # In a production version, deduplicate and order the
    # extreme endpoints of supporting lines around the hull.
    return edges
```

4. Test configurations and correctness

Case 1: Points (0,0), (4,0), (4,3), (0,3), (2,1), (1,2). Expected hull vertices: (0,0), (4,0), (4,3), (0,3); the two inner points are excluded.

Case 2: Points (0,0), (2,2), (4,0), (2,1), (2,–1). The outer vertices are (0,0), (2,2), (4,0), (2,–1). The interior point (2,1) is excluded.

The supporting-line test is justified because a line with all points on one side is a supporting line of the set. The segment between the extreme points on that line lies on the convex boundary. There are $O(n^2)$ candidate pairs and $O(n)$ point checks per pair, so the brute-force test costs $O(n^3)$. Ordering vertices and handling collinear points are required to construct a correct boundary.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

1. Strategy and implementation

Represent each package subset with an n-bit mask: bit i = 1 means package i is selected. Enumerate masks from 0 to 2■–1, sum the selected weights and values, reject subsets exceeding capacity, and retain the feasible subset with maximum value.

```
def exhaustive_knapsack(packages, capacity):
    n = len(packages)
    best_value = best_weight = 0; best_subset = []
    subsets_checked = 0
    for mask in range(1 << n):
        subsets_checked += 1
        weight = value = 0; chosen = []
        for i, (pid, w, v) in enumerate(packages):
            if mask & (1 << i):
                weight += w; value += v; chosen.append(pid)
        if weight <= capacity and value > best_value:
            best_value, best_weight, best_subset = value, weight, chosen
    return best_subset, best_weight, best_value, subsets_checked
```

2. Independently verified small instance

Payload limit = 7. Packages: A (weight 2, value 6), B (3, 8), C (4, 9), D (5, 10). Feasible combinations include A+B (weight 5, value 14), A+C (6, 15), A+D (7, 16), and B+C (7, 17). The optimum is B+C, with total weight 7 and value 17.

3. Candidate growth

Packages (n)	Subsets examined (2■)
5	32
10	1,024
15	32,768
20	1,048,576
25	33,554,432

Each added package doubles the number of subsets. The subset count is O(2■); if each subset is evaluated by scanning all packages, time is O(n·2■). Increasing from 20 to 25 packages increases the candidate count from about one million to over 33 million.

4. Deployment decision

Choose a 1-second response-time target on the actual planning computer. Run tests with increasing n and deploy exhaustive search only up to the largest size that consistently meets the target. Without measured timings on the target machine, a precise maximum size cannot be claimed. For larger instances, consider dynamic programming, branch-and-bound or approximation methods according to the constraints and required guarantees.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: Exhaustive search for humanitarian package selection from Q6.

A. Diagnose the computational bottleneck (2 marks)

The algorithm evaluates all 2■ subsets. The dominant work is repeatedly summing package weights and values for each subset. The candidate count doubles with every added package: n=20 gives 1,048,576 subsets, while n=25 gives 33,554,432. Exponential solution-space growth causes the limitation.

B. Define the design objective (2 marks)

Reduce the number of subsets fully evaluated while preserving the capacity constraint and exact optimal value. Measure states processed, candidate updates and elapsed time.

C. Revised computational strategy (4 marks)

Use dynamic programming (DP) for integer weights and moderate capacity. Let dp[c] be the best value achievable with capacity at most c after processing packages so far. Process each package once and update capacities in descending order so the same package cannot be reused.

```
def dp_knapsack(packages, capacity):
    dp = [0] * (capacity + 1)
    for pid, w, v in packages:
        for c in range(capacity, w-1, -1):
            dp[c] = max(dp[c], dp[c-w] + v)
    return dp[capacity]

packages = [("A",2,6), ("B",3,8), ("C",4,9), ("D",5,10)]
print(exhaustive_knapsack(packages, 7)[2])  # 17
print(dp_knapsack(packages, 7))             # 17
```

DP reuses solutions to smaller capacities rather than re-evaluating every subset. Time is O(nW) and memory is O(W), where W is the integer capacity. This is pseudo-polynomial: very large capacities may make it unsuitable. Reconstructing the selected package IDs requires storing a full table or predecessor decisions.

D. Predict before implementing (1 mark)

Prediction: with moderate W, DP will return the same optimal value while processing far fewer states than exhaustive enumeration. The primary measure is the number of states or candidate operations.

E. Implement and validate (2 marks)

For the four-package example and capacity 7, exhaustive search checks 16 subsets and returns value 17; DP also returns value 17. For n=20 and W=100, DP requires at most about n×(W+1)=2,020 item-capacity updates, compared with 1,048,576 exhaustive subsets. Exact counts depend on weights and implementation.

F. Defend the final design (2 marks)

DP meets the objective for moderate integer capacity because it avoids explicit subset enumeration while preserving the exact optimum. Its costs are dependence on W and additional memory. If capacity is extremely large or weights are non-integers, DP may be impractical or require scaling. Exhaustive search remains useful for very small n and transparent verification. Final deployment should be confirmed with measured timings and operation counts on the target computer.

Appendix – Reproducibility and Submission Checklist

1. Run all code in Python 3 and record actual outputs and execution times where requested.

2. Treat operation counts as primary evidence; execution time is supplementary and varies by device.

3. For Q5, the implementation must directly use the supporting-line test if the instructor requires a brute-force implementation; ensure accepted edges are deduplicated, extreme collinear points retained, and the boundary ordered.

4. Add a plotted convex hull and graphs of input size versus operation count if required.

5. Replace illustrative counts or predictions with the raw outputs from your own runs before submission.